ADVANCED OPERATING SYSTEMS · ASSIGNMENT 3

Peer-to-Peer Distributed File Sharing System

Detailed design and working report, with the algorithm behind each component

Name	Nilkanta Karak
Roll Number	2025201031
Course	Advanced Operating Systems (AOS)
Institute	IIIT Hyderabad
Semester	Monsoon 2025
Language / Platform	C++11, POSIX sockets, pthreads, OpenSSL · Linux

Contents

1. Introduction
2. System Architecture
3. Source Code Organisation
4. Building and Running
5. Core Data Structures
6. Communication Protocols
7. Tracker Algorithms
    1. Startup and role election
    2. Client connection handling
    3. Full-state synchronisation
    4. Incremental update replication
    5. Failover and promotion
8. User and Session Algorithms
9. Group Management Algorithms
10. File Sharing Algorithms
    1. Chunking and hashing (upload)
    2. Peer discovery
    3. Parallel piece download
    4. Piece request and verification
    5. Reassembly and full-file check
    6. Automatic re-seeding
    7. Seeder (uploader) server
    8. Seed persistence
    9. Stop share
    10. Show downloads
11. End-to-End Walkthrough
12. Concurrency and Synchronisation
13. Fault Tolerance and Error Handling
14. Performance and Complexity
15. Limitations and Assumptions
16. Command Reference
17. Glossary

1. Introduction

This project is a distributed peer-to-peer (P2P) file sharing system, similar in spirit to BitTorrent. Users form **groups** and share files inside them. File data never passes through a central server: it moves directly between the users' machines (**peers**). A central **tracker** keeps only metadata: who the users are, which groups exist, which files each group has, and which peers currently hold each file.

1.1 Goals

- **Decentralised data transfer:** file contents move peer-to-peer and the tracker never stores file bytes.
- **Parallelism:** a file is split into fixed 512 KB pieces that are fetched at the same time from several peers.
- **Integrity:** every piece is checked with a SHA-1 hash, and the complete file is checked against a full-file SHA-1.
- **Fault tolerance:** two trackers run as primary and backup, and the backup takes over if the primary dies.
- **Access control:** only accepted members of a group can use that group's files, and group owners approve join requests.
- **Swarm growth:** once a peer finishes downloading a file, it automatically becomes a seeder for that file.

1.2 Key terminology

Term	Meaning in this project
Tracker	Central metadata server. Two instances run: one PRIMARY that serves clients, and one BACKUP that mirrors its state.
Client / Peer	The user program. It is a command shell that talks to the tracker, and it also runs a small server that sends file pieces to other peers.
Piece / Chunk	A 512 KB (524,288-byte) block of a file. The last piece may be shorter.
Seeder	A peer that holds a complete file and serves its pieces to others.
Owner (of a file)	A user registered on the tracker as a holder of a file. Owners who are logged in become the "active peers".

2. System Architecture

The system has three kinds of participant and three separate communication channels.

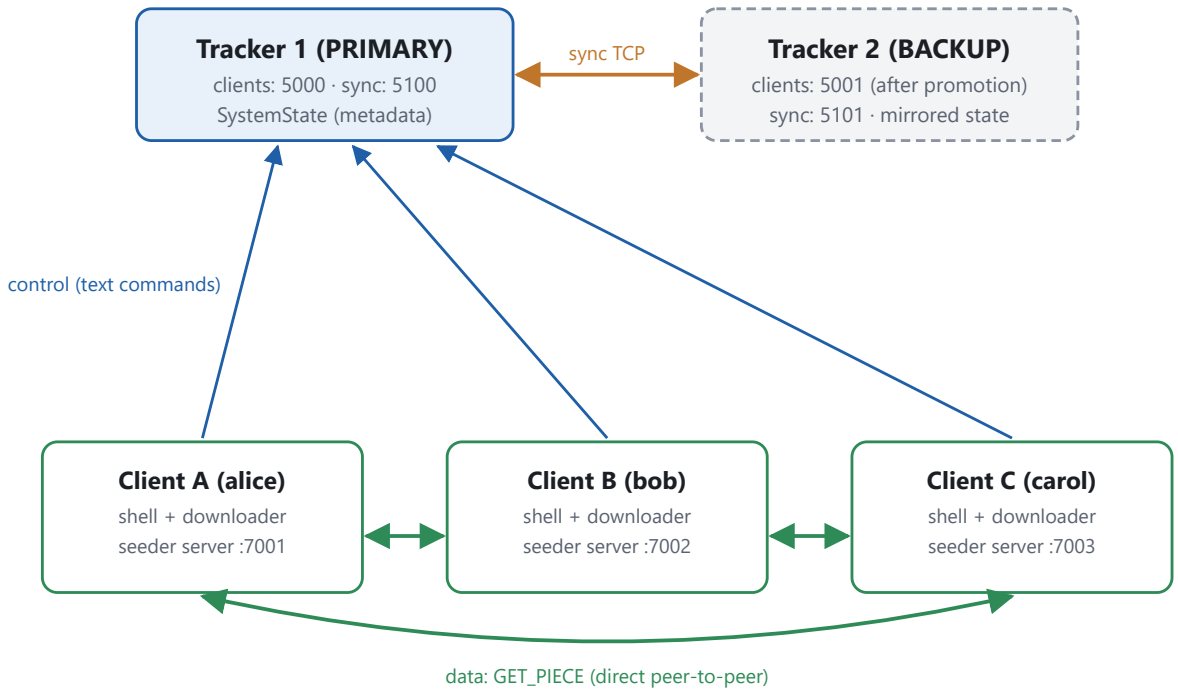


Figure 1: Components and channels. Blue: client-to-tracker control. Green: peer-to-peer data. Orange: tracker-to-tracker replication.

2.1 Components

Component	Role	Main threads
Tracker (PRIMARY)	Accepts client connections. Authenticates users. Manages groups and file metadata. Returns the list of peers for a download. Sends each state change to the backup.	Listener thread; one thread per client; console thread; sync thread
Tracker (BACKUP)	Does not serve clients. Keeps a copy of the primary's state. If the primary disconnects, it promotes itself to primary.	Sync receive thread
Client (shell)	Reads user commands, sends them to the primary tracker and prints the replies.	Main thread
Client (downloader)	Downloads pieces from peers in parallel, verifies them, and writes them into the output file.	1 coordinator + up to 8 workers per download
Client (seeder server)	Listens on the client's own `IP:PORT` and answers `GET_PIECE` requests from other peers.	Accept thread; one thread per request
Common	Protocol constants (`protocol.h`) and a SHA-1 helper (`sha1.cpp`, which wraps OpenSSL).	–

2.2 Ports used

Port	Owner	Purpose
5000 / 5001	Tracker 1 / 2	Client connections. Taken from `tracker_info.txt`.
5100 / 5101	Tracker 1 / 2	Tracker-to-tracker sync. Always the client port + 100. It is set in the `Tracker` constructor.
e.g. 7001, 7002...	Each client	Seeder server. This is the `IP:PORT` given on the client command line.

> The tracker's `main.cpp` also computes `port + 1000` in two variables, `syncPort` and `peerSyncPort`, but nothing uses them. The port that actually carries sync traffic is **port + 100**, as set in `Tracker::Tracker(...)`.

3. Source Code Organisation

File	Responsibility
common/	
`include/protocol.h`	Command names (`create_user`, `login`, ...), reply prefixes (`OK`, `ERR`), `WHOAREYOU` / `ROLE` ... strings, and error code constants.
`src/sha1.cpp`	`computeSHA1(data, len, hex)`: wraps OpenSSL `SHA1()` and returns a 40-character lowercase hex digest.
tracker/	
`src/main.cpp`	Parses `tracker_info.txt`, picks this tracker's own address and its peer's address, starts sync, and starts serving clients if this tracker is PRIMARY.
`src/tracker.cpp`	`Tracker` class: listening socket, thread-per-client loop, line framing, the `processCommand` dispatcher, per-socket sessions, and automatic logout on disconnect.
`src/SystemState.cpp`	All metadata logic: users, groups, join requests, ownership transfer, and file registry. Guarded by a mutex.
`src/sync.cpp`	`TrackerSync`: role election, full-state snapshot (serialise/deserialise), incremental `UPDATE` replication, and promotion.
`tracker_info.txt`	One tracker per line as `IP PORT`, separated by a space (for example `127.0.0.1 5000`).
client/	
`src/main.cpp`	Parses arguments, finds the PRIMARY tracker with `WHOAREYOU`, runs the read-eval-print loop, and handles the reply to each command.
`src/client.cpp`	`Client` class: TCP connection to the tracker, `sendMessage` (loops until every byte is sent), and `receiveMessage` (buffered, newline-terminated).
`src/commands.cpp`	`handleCommand`: checks arguments locally, formats the request, and does client-side bookkeeping (login ordering, `leave_group` / `stop_share` seed cleanup).
`src/uploader.cpp`	`uploadFile`: splits the file into pieces, computes the piece and full-file hashes, sends the metadata, and records the file as seeded.
`src/uploader_server.cpp`	Seeder server: accept loop and `GET_PIECE` handler that uses `pread`. Logs to `uploads.log`.
`src/downloader.cpp`	Parses the download reply, runs the parallel worker pool, requests and verifies pieces, writes them with `pwrite`, checks the full file, re-seeds, and implements `show_downloads`. Logs to `downloads.log`.
`src/seeds.cpp`	Saves and loads the per-user seed list in `~/.p2p_seeds_<user>.txt`.

4. Building and Running

4.1 Prerequisites

You need Linux, g++ with C++11 support, and the OpenSSL development package (`libssl-dev`). Both Makefiles link with `-lcrypto -pthread` . The client also defines `-D_FILE_OFFSET_BITS=64` so that it can handle files larger than 2 GB.

```
sudo apt-get install build-essential libssl-dev
cd tracker && make        # produces tracker/tracker
cd ../client && make      # produces client/client
```

4.2 Tracker configuration

```
# tracker/tracker_info.txt   (IP, space, PORT; one tracker per line)
127.0.0.1 5000
127.0.0.1 5001
```

4.3 Start-up order

```
# Terminal 1 – whichever tracker starts first becomes PRIMARY
./tracker tracker_info.txt 1
# Terminal 2 – finds the running peer and becomes BACKUP
./tracker tracker_info.txt 2
# Terminals 3, 4 … – each client gets its own seeder IP:PORT
./client 127.0.0.1:7001 ../tracker/tracker_info.txt
./client 127.0.0.1:7002 ../tracker/tracker_info.txt
```

> Roles are decided by **start order**, not by the tracker number. If tracker 2 is started first, tracker 2 becomes PRIMARY. Clients find the primary with `WHOAREYOU` (Algorithm 7.2), so either order works.

5. Core Data Structures

5.1 Tracker: `SystemState`

```
struct User         { string userId; string passwordHash; bool loggedIn; };
struct Group        { string groupId; string ownerId;
                      set<string> members;          // accepted members, including the owner
                      set<string> pendingRequests;  // users waiting for approval
                      vector<string> memberOrder; }; // join order, used to pick a new owner
struct FileMetadata { string groupId, fileName; size_t fileSize, numChunks;
                      string fullHash; vector<string> pieceHashes;
                      set<string> owners;          // userIds holding the file
                      set<string> activePeers; };  // "ip:port" of owners who are logged in

class SystemState {
  map<string, User>                          users;         // userId  -> User
  map<string, Group>                         groups;        // groupId -> Group
  map<string, map<string, FileMetadata>>     groupFiles;    // groupId -> fileName -> meta
  map<string, string>                        userSessions;  // userId  -> "ip:port" of seeder
};
```

Every `SystemState` method takes one file-scope `static mutex` before touching these maps, so tracker threads can call it at the same time. The containers are `std::map` and `std::set` (balanced trees), so lookups cost O(log n) and listings come out in alphabetical order.

Why `userSessions` matters: a file stores its owners as *user IDs*. At download time, the tracker converts each owner ID into the seeder address that user gave at login. Owners who are logged out have no entry in `userSessions`, so they drop out of the peer list automatically.

5.2 Tracker: per-connection session

```
struct ClientSession { string userId; bool loggedIn; string peerAddr; };
unordered_map<int /*socket fd*/, ClientSession> sessions;   // guarded by sessionMutex
```

The tracker identifies a request's user by the **socket it arrived on**, not by anything in the message text. After `login`, every later command on that socket runs as that user.

5.3 Client-side structures

Structure	Where	Purpose
`map<user, map<fileName, SeedInfo{path, groupId}>> userSeeds`	downloader.cpp (global)	Files this process can serve. The seeder server looks up the local path here.
`DownloadMetadata{fileName, fileSize, numChunks, fullHash, groupId, pieceHashes, peers}`	downloader.h	Parsed form of the tracker's `OK DOWNLOAD ...` reply.
`DownloadContext{workQueue, queueMutex, rng, meta, fd, retryCount, pieceDone, piecesCompleted}`	downloader.cpp	Shared state for all worker threads of one download.
`map<fileName, DownloadContext*> activeDownloads`	downloader.cpp	Downloads in progress. `show_downloads` reads it.

`map<user, map<fileName, CompletedInfo>> completedDownloads`	downloader.cpp	Finished downloads, shown as `[C]`.
`~/.p2p_seeds_<user>.txt`	disk	Seed list that survives a restart. Each line is `fileName path groupId`.

6. Communication Protocols

All three channels use **TCP**. Control messages are **plain text lines ending in `\n`**, with fields separated by spaces. File pieces are sent as raw bytes.

6.1 Message framing

- **Tracker receive:** each client thread calls `recv()` into a 4 KB buffer and appends the bytes to a `leftover` string. It then pulls out every complete line, strips `\r`/`\n`, and processes each one. A long message, such as an upload with 200 piece hashes, can arrive over several `recv` calls and is still read correctly.
- **Tracker send:** it appends `\n` if missing and repeats `send()` until every byte has gone (partial sends are handled).
- **Client receive:** `Client::receiveMessage` reads until the buffer contains at least one `\n`, then returns the whole buffer as one reply. This is how multi-line replies such as `list_groups` arrive as a single message.
- **Sync channel:** reads one byte at a time up to `\n`. A full snapshot is sent as one line: newlines inside it are replaced by `;` before sending and put back on arrival.

6.2 Client ↔ Tracker commands

User types	Wire request	Success reply
—	`WHOAREYOU`	`ROLE PRIMARY` | `ROLE BACKUP`
`create_user u p`	`create_user u p`	`OK User created: u`
`login u p`	`login u p ip:port` (client appends its seeder address)	`OK Logged in as u`
`logout`	`logout`	`OK Logged out u`
`create_group g`	`create_group g`	`OK Group created: g`
`list_groups`	`list_groups`	`OK Groups:\n - g1\n - g2`
`join_group g`	`join_group g`	`OK Join request sent for group g`
`list_requests g`	`list_requests g`	`OK Pending requests for group g:\n - u2`
`accept_request g u`	`accept_request g u`	`OK User u added to group g`
`leave_group g`	`leave_group g`	`OK u left group g`
`upload_file g path`	`upload_file g name size nChunks fullHash h1 … hN`	`OK File name uploaded in group g by u`
`list_files g`	`list_files g`	`OK Files in group g:\n - f`
`download_file g f dest`	`download_file g f dest`	`OK DOWNLOAD name size nChunks fullHash g h1 … hN PEER ip:port PEER ip:port …`
`stop_share g f`	`stop_share g f`	`OK u stopped sharing f in group g`
`show_downloads`	*(local only, nothing sent)*	Printed from local state
`exit`	*(local only)*	Client closes the connection

Errors come back as `ERR <reason>`, for example `ERR Must be logged in` or `ERR Group does not exist`. Some errors include a numeric code, for example `ERR 500 Unknown command: x`.

6.3 Tracker ↔ Tracker (sync channel)

Message	Direction	Meaning
`SYNC_REQUEST`	Backup → Primary	"Send me your whole state."
`SYNC_RESPONSE <records separated by ;>`	Primary → Backup	Full snapshot (see 7.3).
`UPDATE CREATE_USER u p`	Primary → Backup	Replays one state change that already succeeded on the primary (see 7.4).
`UPDATE GROUP_CREATE g owner`		
`UPDATE JOIN_GROUP g u`		
`UPDATE ACCEPT_REQUEST g owner u`		
`UPDATE LEAVE_GROUP g u`		
`UPDATE LOGOUT u`		
`UPDATE UPLOAD_FILE g u name size n fullHash h1…hN`		
`UPDATE STOP_SHARE g name u`		

6.4 Peer ↔ Peer (data channel)

```
Downloader  ──►  "GET_PIECE <fileName> <pieceIndex>\n"
Seeder      ──►  exactly expectedSize raw bytes of that piece, then closes the connection
             or  "ERR <reason>\n", then closes
                 (Invalid request | File not available | Could not stat file |
                  Invalid piece index | Could not open file | Read failed | Short read)
```

Each piece uses **one short-lived TCP connection**. The downloader already knows how many bytes to expect from the metadata, so it needs no length header. It reads until it has `expectedSize` bytes or the seeder closes the connection.

7. Tracker Algorithms

7.1 Tracker startup and role election

Files: tracker/src/main.cpp, sync.cpp: `TrackerSync::start`, `connectToPeer`

Idea: there is no voting. A tracker tries to connect to its peer's sync port. If a tracker is already listening there, a primary exists and this tracker becomes the backup. If the connection is refused, nobody is running yet and this tracker becomes the primary.

```
ALGORITHM TrackerStartup(infoFile, n)
  trackers ← parse lines "IP PORT" from infoFile
  (myIP, myPort)     ← trackers[n-1]
  (peerIP, peerPort) ← trackers[other]
  syncPort ← myPort + 100;  peerSyncPort ← peerPort + 100

  sock ← TCP connect(peerIP, peerSyncPort)
  IF connect fails THEN                          // no one is running
      role ← PRIMARY
      listen on syncPort                           // wait for a future backup
      spawn thread AcceptBackupConnection()
      start client listener on myPort              // Tracker::start()
  ELSE                                             // a primary is running
      role ← BACKUP
      send "SYNC_REQUEST"                          // ask for a full snapshot
      spawn thread HandlePeerUpdates()             // apply snapshot and later UPDATEs
      // a BACKUP does NOT open its client port
  loop forever (sleep)
```

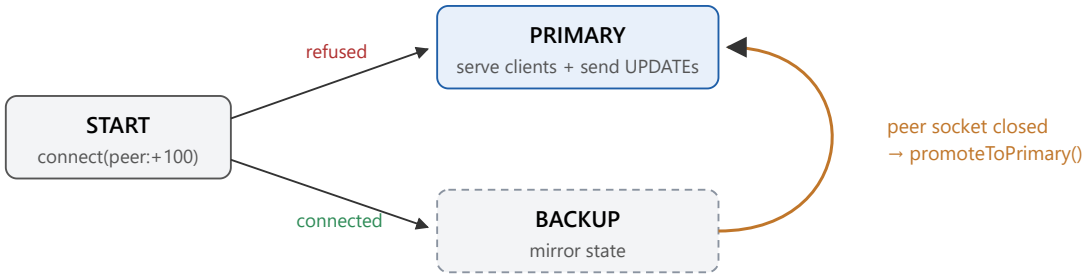


Figure 2: Tracker role state machine.

Complexity: one connection attempt, O(1).

7.2 Client finding the primary tracker (`WHOAREYOU`)

File: client/src/main.cpp

```
ALGORITHM FindPrimary(trackerList)
  FOR each (ip, port) in trackerList IN ORDER
      IF TCP connect(ip, port) fails THEN continue    // down, or a BACKUP (its client port is
closed)
      send "WHOAREYOU"
      IF reply contains "ROLE PRIMARY" THEN
          keep this connection; RETURN success
      ELSE disconnect
  RETURN failure  → "Could not connect to any PRIMARY tracker"
```

A backup never opens its client port, so connecting to it simply fails. The `WHOAREYOU` check is an extra safeguard. After a failover the old backup is primary on its own port, and a restarted client reaches it by moving on to the next entry in the list.

7.3 Handling client connections (thread-per-client)

File: tracker/src/tracker.cpp: `listenForClients`, `handleClient`, `processCommand`

```
ALGORITHM ListenForClients()
  WHILE running
      s ← accept(server_fd)
      spawn detached thread HandleClient(s)

ALGORITHM HandleClient(s)
  sessions[s] ← empty session;  leftover ← ""
  LOOP
      bytes ← recv(s, 4096)
      IF bytes = 0 or error THEN break                 // client left
      leftover += bytes
      WHILE leftover contains '\n'
          line ← cut first line; trim \r\n; skip if empty
          reply ← (line = "WHOAREYOU") ? role string : ProcessCommand(line, s)
          send all bytes of reply + '\n'
  // ---- cleanup: automatic logout ----
  IF sessions[s].loggedIn THEN
      SystemState.logoutUser(user); sync.sendUpdate("LOGOUT user")
  erase sessions[s]; close(s)

ALGORITHM ProcessCommand(line, s)
  action ← first word
  check arguments                        → "ERR Missing ..."
  IF action needs login AND NOT sessions[s].loggedIn → "ERR Must be logged in"
  user ← sessions[s].userId
  ok ← SystemState.<operation>(…, user, errMsg)
  IF NOT ok THEN RETURN "ERR " + errMsg
  IF operation changes state THEN sync.sendUpdate(<UPDATE record>)
  RETURN "OK …"
```

Automatic logout means that if a client crashes or exits, the tracker marks the user logged out and removes their seeder address. Other peers will therefore not be sent to a dead address.

7.4 Full-state synchronisation (snapshot)

File: sync.cpp: `serializeSystemState`, `deserializeSystemState`

When a backup (re)joins, it knows nothing, so the primary sends its entire state as a list of text records.

```
ALGORITHM Serialize(state)
  FOR each user u:        emit "USER    u.id u.password u.loggedIn(0/1)"
  FOR each group g:       emit "GROUP   g.id g.owner"
      FOR m in g.members:         emit "MEMBER  g.id m"
      FOR r in g.pendingRequests: emit "REQUEST g.id r"
      FOR o in g.memberOrder:     emit "ORDER   g.id o"
  FOR each group g, file f:
      emit "FILE g f.name f.size f.numChunks f.fullHash"
      FOR h in f.pieceHashes:   emit "PIECE  g f.name h"     // order preserved
      FOR o in f.owners:        emit "OWNER  g f.name o"
      FOR p in f.activePeers:   emit "ACTIVE g f.name p"
  IF nothing emitted THEN emit "EMPTY"
  replace every '\n' with ';'  → send "SYNC_RESPONSE " + text

ALGORITHM Deserialize(text)
  replace ';' with '\n'; state ← empty SystemState
  FOR each line: switch on first word and insert into the matching map/set/vector
```

Worked example. State: user *alice*, group *g1* owned by alice, and file *a.txt* (3 pieces) in g1. The snapshot line is:

```
SYNC_RESPONSE USER alice pw 1;GROUP g1 alice;MEMBER g1 alice;ORDER g1 alice;FILE g1 a.txt
1300000 3 9f…e1;PIECE g1 a.txt 1a…;PIECE g1 a.txt 2b…;PIECE g1 a.txt 3c…;OWNER g1 a.txt alice;
```

Complexity: O(U + G·M + F·P), a linear pass over all the metadata.

7.5 Incremental update replication

File: sync.cpp: `sendUpdate`, `handlePeerUpdates`, `applyUpdate`

After the snapshot, the backup stays up to date through **operation-log replication**. The primary first applies a change to its own state. Only if that succeeds does it send the same operation to the backup, which then runs the same `SystemState` method.

```
ALGORITHM SendUpdate(op)                      // PRIMARY only
  IF role ≠ PRIMARY THEN return
  append op to updateLog (under logMutex)
  IF backup connected THEN send "UPDATE " + op

ALGORITHM HandlePeerUpdates()                 // runs in a thread on BOTH trackers
  LOOP
    msg ← read line from sync socket
    IF socket closed:
        IF I am PRIMARY: close; spawn AcceptBackupConnection(); return    // wait for a new
backup
        ELSE:            PromoteToPrimary(); return                       // primary died
    IF msg = "SYNC_REQUEST" and I am PRIMARY: send snapshot
    IF msg starts "SYNC_RESPONSE" and I am BACKUP: Deserialize(rest)
    IF msg starts "UPDATE": ApplyUpdate(rest)

ALGORITHM ApplyUpdate(op)
  CREATE_USER u p        → users[u] ← User(u,p)
  GROUP_CREATE g o       → groups[g] ← {owner o, members {o}, order [o]}
  JOIN_GROUP g u         → joinGroup(g,u)
  ACCEPT_REQUEST g o u   → acceptRequest(g,o,u)
  LEAVE_GROUP g u        → leaveGroup(g,u)          // same ownership-transfer logic
  LOGOUT u               → logoutUser(u)
  UPLOAD_FILE …          → uploadFile(…) and add u to owners
  STOP_SHARE g f u       → stopShare(g,u,f)
```

Both trackers run the **same deterministic code** on the same inputs in the same order (one TCP stream keeps the order). Their states therefore stay identical without shipping whole objects.

> **Not replicated:** `login`, which means the `userSessions` address map. The backup cannot use these addresses, because after a failover every client must reconnect and log in again, and that login recreates the session on the new primary.

7.6 Failover and promotion

File: sync.cpp: `promoteToPrimary`, `acceptBackupConnection`

```
ALGORITHM PromoteToPrimary()                  // on BACKUP when the sync socket closes
  IF role = PRIMARY THEN return
  role ← PRIMARY
  close old sync socket
  Tracker.start()                               // open client port and start the listener
  listen on own syncPort
  spawn AcceptBackupConnection()                // the old primary can rejoin as BACKUP
```

Timeline example:

1. T1 starts and becomes PRIMARY. T2 starts, connects to 5100, becomes BACKUP and receives the snapshot.
2. Users work through T1. Every change is also applied on T2.
3. T1 is killed. T2's `recv()` returns 0 → "*Peer disconnected, promoting to PRIMARY*" → T2 opens port 5001.
4. Clients connected to T1 see "*Connection closed by tracker*" and exit. When restarted, they skip T1 (refused), reach T2 (`ROLE PRIMARY`) and log in again. Their accounts, groups and files are still there,

and saved seeds are reloaded.

5. T1 restarts, connects to T2's sync port 5101, becomes BACKUP and receives a fresh snapshot.

8. User and Session Algorithms

8.1 Create user

```
IF userId ∈ users THEN ERR "User already exists"
users[userId] ← User(userId, password, loggedIn = false)
sync "CREATE_USER userId password"
```

No login is needed. Cost O(log U).

8.2 Login: client and tracker together

Files: client/src/commands.cpp, client/src/main.cpp, tracker/src/SystemState.cpp

A login connects the user's identity to a network address where other peers can fetch pieces. The order of steps matters, so that the tracker never advertises an address that nobody is serving.

```
CLIENT  Login(u, p)
  1. IF this client already has a logged-in user → refuse ("run logout first")
  2. StartUploaderServer(myIP, myPort, u)            // seeder is ready BEFORE the tracker is
told
        IF it fails → refuse login (tracker is never told)
  3. send "login u p myIP:myPort"; pendingLoginUser ← u
  4. reply ← receive
        IF reply ≠ "OK Logged in as u":  StopUploaderServer(); clear userId   // roll back
        ELSE: userSeeds[u] ← LoadSeeds(u)            // resume seeding files from earlier
sessions

TRACKER LoginUser(u, p, addr)
  IF u ∉ users           → ERR "User does not exist"
  IF password mismatch   → ERR "Incorrect password"
  users[u].loggedIn ← true
  userSessions[u] ← addr                             // used later for peer discovery
  sessions[socket] ← {u, loggedIn, addr}
```

Why this order: if the tracker were told about the address first and the seeder failed to start, every download request sent to that address would get `ERR File not available`. The client also refuses a second login in the same process. One process has only one seeder port, so it cannot serve two users.

8.3 Logout (explicit and automatic)

```
TRACKER Logout(socket)
  IF no logged-in session on socket → ERR "No active login session"
  users[u].loggedIn ← false; erase userSessions[u]; clear session
  sync "LOGOUT u"
CLIENT on "OK Logged out": StopUploaderServer(); clear userId
```

Automatic logout happens in `HandleClient` cleanup (7.3) whenever the TCP connection drops. The user's files keep them as an *owner*, but the user is no longer an *active peer*, so downloaders are not sent to them until they log in again.

9. Group Management Algorithms

9.1 Create group

```
IF gid ∈ groups → ERR "Group already exists"
groups[gid] ← { owner = u, members = {u}, pendingRequests = {}, memberOrder = [u] }
sync "GROUP_CREATE gid u"
```

9.2 Join request, list requests and accept (owner-approved membership)

```
JoinGroup(gid, u)
  IF gid ∉ groups                   → ERR "Group does not exist"
  IF u ∈ members                    → ERR "Already a member of this group"
  IF u ∈ pendingRequests            → ERR "Join request already sent"
  pendingRequests.insert(u); sync "JOIN_GROUP gid u"

ListRequests(gid, caller)
  IF caller ≠ owner                 → ERR "Only the group owner can view requests"
  RETURN pendingRequests (sorted)

AcceptRequest(gid, caller, u)
  IF caller ≠ owner                 → ERR "Only the group owner can accept requests"
  IF u ∉ pendingRequests            → ERR "No such pending request from u"
  pendingRequests.erase(u); members.insert(u); memberOrder.push_back(u)
  sync "ACCEPT_REQUEST gid caller u"
```

This is a two-step handshake. Nobody enters a group without the owner's approval. Each accepted member is appended to `memberOrder`, which records seniority for the next algorithm.

9.3 Leave group, with ownership transfer and file cleanup

File: SystemState.cpp: `leaveGroup`

```
ALGORITHM LeaveGroup(gid, u)
  IF gid ∉ groups                       → ERR
  IF u ∉ members AND u ≠ owner          → ERR "You are not a member of this group"

  // Step 1: remove u as owner of every file in this group
  FOR each file f in groupFiles[gid]
      f.owners.erase(u)
      IF f.owners is empty THEN mark f for removal   // nobody holds it any more
  erase marked files; IF group has no files THEN erase groupFiles[gid]

  // Step 2: membership and ownership
  IF u = owner THEN
      newOwner ← first m in memberOrder with m ≠ u and m ∈ members    // most senior member
      IF newOwner exists THEN owner ← newOwner; remove u from members and memberOrder
      ELSE erase the whole group                                      // last member left
  ELSE remove u from members and memberOrder
  sync "LEAVE_GROUP gid u"

CLIENT side: remove every userSeeds entry whose groupId = gid; rewrite the seeds file
```

Example: g1 was created by alice, who then accepted bob and later carol. So `memberOrder = [alice, bob, carol]`. When alice leaves, **bob** becomes owner, because he is the earliest member still in the group. If bob and carol had already left, g1 would be deleted.

Complexity: O(F log F + M), where F is the number of files in the group and M the number of members.

10. File Sharing Algorithms

10.1 Upload: chunking and hashing

File: client/src/uploader.cpp: `uploadFile` ; tracker: `SystemState::uploadFile`

"Uploading" does **not** send the file to the tracker. It publishes a *description* of the file: its size, a fingerprint of every piece, and a fingerprint of the whole file. The uploader then becomes a seeder.

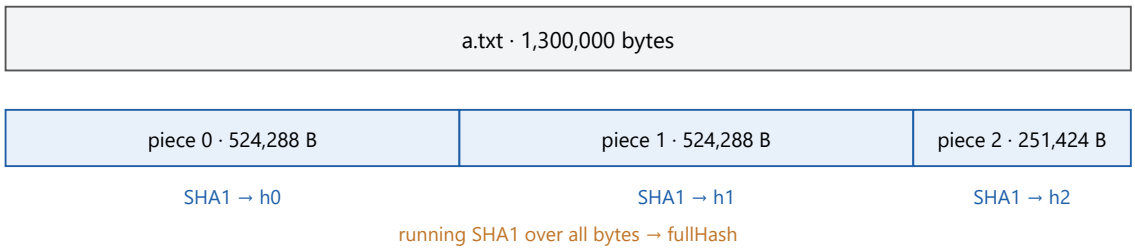


Figure 3: Splitting a file into pieces. numChunks = ⌈size / 524,288⌉. The last piece holds the remainder.

```
ALGORITHM UploadFile(path, gid)                      // client
  size ← stat(path).st_size
  pieceHashes ← [];  SHA1_Init(fullCtx)
  WHILE (n ← read(fd, buf, 512 KB)) > 0              // one pass over the file
      pieceHashes.append( hex(SHA1(buf[0..n))) )     // per-piece fingerprint
      SHA1_Update(fullCtx, buf[0..n))                 // feed the whole-file hash
  fullHash ← hex(SHA1_Final(fullCtx))
  name ← basename(path)
  send "upload_file gid name size |pieceHashes| fullHash h0 h1 … hN-1"
  seeds ← LoadSeeds(user); seeds[name] ← {path, gid}; SaveSeeds(user, seeds)
  userSeeds[user][name] ← {path, gid}                // the seeder can now serve it

ALGORITHM SystemState.UploadFile(gid, u, name, …)   // tracker
  IF gid ∉ groups → ERR
  IF name already in groupFiles[gid] THEN owners.insert(u)   // another copy of the same file
  ELSE groupFiles[gid][name] ← new FileMetadata{…, owners = {u}}
  sync "UPLOAD_FILE gid u name size n fullHash h0…"
```

Why two levels of hashing? Piece hashes let the downloader reject one bad piece and fetch it again from another peer, without restarting the whole download. The full hash is a final end-to-end check that the pieces were joined in the right order and nothing is missing.

Complexity: O(size) disk reads and hashing, done once. The message has 40·N hex characters of hashes, for example about 8 KB for a 100 MB file (200 pieces).

10.2 Peer discovery (tracker side of `download_file`)

File: SystemState.cpp: `getFileMetadata` ; tracker.cpp

```
ALGORITHM GetFileMetadata(gid, name)
  IF gid ∉ groups or name ∉ groupFiles[gid] → ERR "File not found in group gid"
  meta ← copy of groupFiles[gid][name]
  meta.activePeers ← { userSessions[o] : o ∈ meta.owners AND o is logged in }
  reply "OK DOWNLOAD name size n fullHash gid h0 … hN-1  PEER ip:port  PEER ip:port …"
```

The peer list is computed **at request time**, so it only contains owners who are logged in right now. A logged-out owner is left out automatically.

10.3 Parallel piece download (work-queue and thread pool)

File: client/src/downloader.cpp: `startDownload`, `downloadWorker`

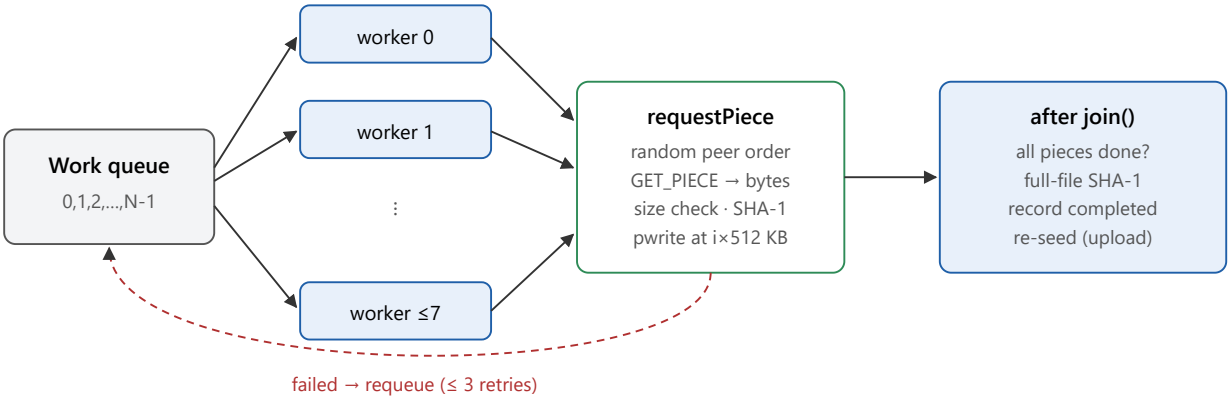


Figure 4: Download pipeline for one file.

```
ALGORITHM StartDownload(meta, destPath)     // returns at once; runs in background
  ctx.workQueue ← [0, 1, …, N-1];  ctx.pieceDone ← [false]*N;  ctx.rng ← random seed
  activeDownloads[meta.fileName] ← ctx
  spawn detached COORDINATOR thread:
      fd ← open(destPath, WRONLY|CREAT|TRUNC)
      ftruncate(fd, meta.fileSize)           // pre-size file: pieces land anywhere
      T ← min(8, N);  start T threads of DownloadWorker; join all
      close(fd)
      IF piecesCompleted ≠ N  → print "k of N pieces unavailable"; delete file
      ELSE IF NOT VerifyFullFile(meta, destPath) → delete file
      ELSE record completed; spawn ReSeed thread (10.6)
      remove from activeDownloads; free ctx

ALGORITHM DownloadWorker(ctx)
  LOOP
      LOCK queue:  IF queue empty → RETURN
                   i ← pop();  IF pieceDone[i] → continue
      peers ← shuffle(meta.peers, ctx.rng)        // random peer order spreads the load
      FOR each peer p in peers
          IF RequestPiece(p, i) THEN              // see 10.4
              LOCK: IF NOT pieceDone[i] THEN pieceDone[i] ← true; piecesCompleted++
              success; break
      IF NOT success THEN
          LOCK: retry[i]++
                IF retry[i] ≤ 3 THEN push i back to queue    // try again later
                ELSE give up on piece i (logged)
```

DESIGN POINTS

- **Load balancing:** each piece shuffles the peer list, so on average the requests are spread evenly over all seeders. With S seeders, each serves about N/S pieces.
- **Dynamic scheduling:** workers take the next piece from a shared queue instead of receiving fixed ranges. A fast worker simply handles more pieces.

- **Retry policy:** in each attempt the worker tries *every* peer. A piece that still fails goes to the back of the queue, up to 3 more times, so each piece gets at most 4 rounds × S peers.
- **Piece order:** pieces are handed out in queue order (0, 1, 2, ...) and the peer is chosen at random. This is not BitTorrent's rarest-first strategy. Every peer returned here holds the complete file, so rarity does not matter.
- **Non-blocking shell:** `startDownload` returns immediately, so the user can keep typing commands, including `show_downloads`, while the file downloads.

10.4 Requesting and verifying one piece

File: downloader.cpp: `requestPiece`, `expectedPieceSize`

```
expectedSize(i) = 524,288                                if i < N-1
                = fileSize − (N−1)·524,288               if i = N-1   (last piece)

ALGORITHM RequestPiece(peer "ip:port", name, i, expectedHash, expectedSize, fd)
  s ← socket; set SO_RCVTIMEO = 8 s                     // a silent peer cannot hang the worker
  connect(ip, port)                          ELSE fail
  send "GET_PIECE name i\n"
  buf ← read until expectedSize bytes OR connection closed
  IF buf starts with "ERR" → log the reason; fail         // peer refused
  IF |buf| ≠ expectedSize  → fail                          // cut short
  IF hex(SHA1(buf)) ≠ expectedHash → fail                   // corrupt or wrong data
  pwrite(fd, buf, expectedSize, offset = i × 524,288)     // write at the piece's position
  succeed
```

Why `pwrite`? It writes at an explicit offset and does not move a shared file pointer. Up to 8 threads can therefore write different pieces into the same file descriptor at the same time with no lock, because each piece has its own byte range.

Example: piece 2 of the 1,300,000-byte file is 251,424 bytes and is written at offset 1,048,576.

10.5 Reassembly and full-file verification

File: downloader.cpp: `verifyFullFile`

```
ALGORITHM VerifyFullFile(meta, path)
  SHA1_Init(ctx)
  WHILE (n ← read(path, 512 KB)) > 0 : SHA1_Update(ctx, n bytes)   // streaming, constant memory
  IF hex(SHA1_Final(ctx)) = meta.fullHash → success  ELSE → mismatch (file deleted)
```

Because `ftruncate` made the file its final size up front, and every piece was written at its own offset, the file is already complete once all workers finish. No separate "join the pieces" step is needed. The streaming hash reads the file in 512 KB blocks, so memory use stays constant even for very large files.

10.6 Automatic re-seeding after a download

```
ALGORITHM ReSeed(destPath, gid)               // detached thread after a verified download
  LOCK clientUploadMutex
  UploadFile(destPath, gid)                     // same as 10.1: hash, send metadata, save seed
  read the tracker's reply and print it
  userSeeds[user][basename(destPath)] ← {destPath, gid}
```

On the tracker, the file name already exists in the group, so `uploadFile` just adds this user to `owners`. The next downloader gets one more `PEER`. Popular files therefore gain more seeders over time. This is the key scalability property of P2P sharing.

10.7 Seeder (uploader) server

File: client/src/uploader_server.cpp

```
ALGORITHM StartUploaderServer(ip, port, user)
  IF already running for the same user → OK
  IF running for a different user → StopUploaderServer() first   // never serve as the wrong
user
  bind + listen(ip:port, backlog 16);  serverUserId ← user
  ACCEPT THREAD: WHILE running: s ← accept(); spawn thread HandlePeerRequest(s, user)

ALGORITHM HandlePeerRequest(s, user)
  read one request; parse "GET_PIECE name i"          ELSE "ERR Invalid request"
  path ← userSeeds[user][name].path (under lock)      ELSE "ERR File not available"
  size ← stat(path)                                     ELSE "ERR Could not stat file"
  offset ← i × 524,288;  IF offset ≥ size               → "ERR Invalid piece index"
  len ← min(524,288, size − offset)
  pread(path, len, offset)                              ELSE "ERR Read failed" / "Short read"
  send all len bytes; close(s)

ALGORITHM StopUploaderServer()
  running ← false; shutdown+close listening socket (unblocks accept); join accept thread
```

Each request gets its own thread, so one seeder can serve many downloaders, and many pieces to one downloader, in parallel. `pread` is used here for the same reason as `pwrite`: concurrent reads of one file at different offsets do not interfere.

10.8 Seed persistence across restarts

File: client/src/seeds.cpp

```
SeedsFile(user) = $HOME/.p2p_seeds_<user>.txt     // one line per file: "fileName path groupId"
SaveSeeds(user, map): truncate the file and write every entry
LoadSeeds(user):      read the whole file, split by '\n', parse 3 fields per line
```

The tracker remembers that a user *owns* a file even while they are logged out. When the user logs in again, the client reloads this file into `userSeeds`, and the seeder can immediately serve the files the tracker expects it to have.

10.9 Stop sharing

```
TRACKER StopShare(gid, u, name)
  IF group/file missing or u ∉ owners → ERR
  owners.erase(u); activePeers.erase(userSessions[u])
  IF owners empty → remove file from group
  sync "STOP_SHARE gid name u"
CLIENT: userSeeds[u].erase(name); remove from seeds file; SaveSeeds
```

10.10 Show downloads

```
FOR each ctx in snapshot(activeDownloads):
    print "[D] gid fileName : done/total pieces (percent%)"     // piecesCompleted read under
queueMutex
FOR each entry in completedDownloads[user]:
    print "[C] gid fileName"
IF nothing → "No downloads for user"
```

This command runs entirely on the client and sends nothing to the tracker. It first copies the active-download map under a lock, then reads each download's counter under that download's own lock, so it never blocks a running download for long.

11. End-to-End Walkthrough

Scenario: alice shares `a.txt` (1,300,000 bytes) in group `g1`, and bob downloads it.

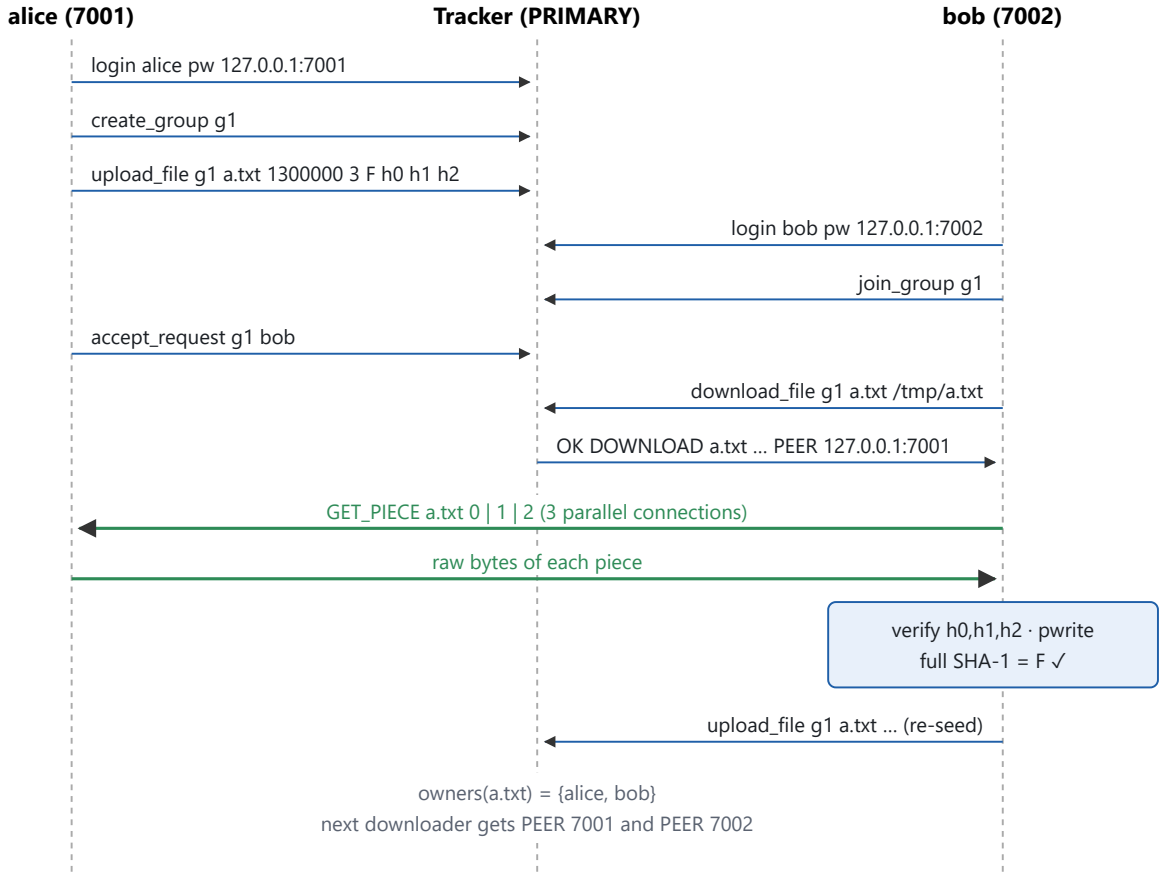


Figure 5: Message sequence for one share-and-download cycle. Blue lines are sent over the control channel and green lines over the peer-to-peer data channel.

1. **alice logs in:** her client starts its seeder on 7001, then sends `login`. The tracker stores `userSessions[alice] = 127.0.0.1:7001`.
2. **alice creates g1**, so alice is the owner and `memberOrder = [alice]`. The backup receives `UPDATE GROUP_CREATE g1 alice`.
3. **alice uploads a.txt:** 3 pieces are hashed locally and the metadata goes to the tracker. `owners = {alice}`. The seed file `~/.p2p_seeds_alice.txt` gains the line `a.txt /home/…/a.txt g1`.
4. **bob logs in and requests to join** g1. alice sees him in `list_requests g1` and accepts, so bob becomes a member.
5. **bob downloads:** the tracker converts owners {alice} into peers {127.0.0.1:7001}. bob's client starts min(8, 3) = 3 workers. Each one requests a piece from alice's seeder, checks its hash, and writes it at its offset.
6. **Verification:** all 3 pieces arrive. The streaming SHA-1 equals `F`, and the client prints `Download complete: /tmp/a.txt`.
7. **Re-seed:** bob's client registers itself as an owner. Anyone who downloads next can fetch pieces from both alice and bob.

12. Concurrency and Synchronisation

Lock	Process	Protects
`stateMutex` (file-static in SystemState.cpp)	Tracker	All `SystemState` maps during every public operation except `stopShare`.
`sessionMutex`	Tracker	The `sessions` map (socket → user).
`logMutex` (TrackerSync)	Tracker	The `updateLog` vector of replicated operations.
`atomic<bool> running`	Tracker, Client	Stop flags for the accept loops. A lock-free read is safe across threads.
`seedingFilesMutex`	Client	`userSeeds` and `completedDownloads`.
`downloadsMutex`	Client	The `activeDownloads` registry.
`ctx->queueMutex` (one per download)	Client	Work queue, `pieceDone`, `retryCount` and `piecesCompleted` for that download.
`clientUploadMutex`	Client	Makes re-seed uploads run one at a time.
`ioMutex`	Client	Printing the prompt and reading a line from stdin.
`logMutex` (downloader / uploader)	Client	Appends to `downloads.log` / `uploads.log`.

Lock-free by design

- **File writes:** `pwrite` at a different offset for each piece means workers never share a file position, so they need no lock.
- **File reads on the seeder:** `pread` opens its own file descriptor for each request, so there is no shared state.
- **Short critical sections:** workers hold the queue lock only to pop an index or update counters. Networking and hashing happen outside the lock.

Thread model summary

Process	Threads
Tracker	main (sleeps) · sync receive · backup accept · client listener · 1 per client · console (`quit` / `exit`)
Client	main shell · seeder accept · 1 per incoming piece request · 1 coordinator + ≤ 8 workers per download · 1 re-seed thread per finished download

13. Fault Tolerance and Error Handling

Failure	How the system reacts
Primary tracker crashes	The backup sees its sync socket close, promotes itself, and opens its client port (7.6). All metadata is still there.
Backup tracker crashes	The primary logs "Backup disconnected" and goes back to waiting for a new backup. Clients are not affected.
Client exits or crashes	The tracker's thread for that client sees EOF and logs the user out automatically. The user's address disappears from peer lists.
A seeder is unreachable	`connect()` fails and the worker tries the next peer in its shuffled list.
A seeder hangs	The 8-second `SO_RCVTIMEO` ends the read. The piece fails on that peer and the next peer is tried.
A seeder no longer has the file	The seeder replies `ERR File not available`, the downloader logs the reason and tries another peer.
Corrupted or truncated piece	The size check or the SHA-1 check fails. The piece is thrown away and fetched again (up to 3 requeues).
A piece cannot be fetched from anyone	After the retries run out, the download reports how many pieces were missing and deletes the partial file.
The assembled file does not match	The full-file SHA-1 differs, so the output file is deleted and the failure is logged.
Destination cannot be created	`open` / `ftruncate` fails, which is logged, and the download is abandoned cleanly.
Interrupted system calls	`EINTR` is retried in the send, receive and accept loops.
Partial `send()`	Every sender loops until all bytes are written.

Logs for debugging: the client writes `downloads.log` (every piece, peer, retry and verification) and `uploads.log` (every piece it serves or refuses) in its working directory. The tracker prints every received command and every sync event to stdout.

14. Performance and Complexity

Operation	Time complexity	Notes
create_user / login / create_group / join / accept	O(log U) or O(log G)	Tree-map lookups.
list_groups / list_files / list_requests	O(G), O(F) or O(R)	Output is sorted by name.
leave_group	O(F log F + M)	Scans the group's files, then the member order.
upload (client)	O(S) for S bytes	One sequential read, two SHA-1 streams.
download_file (tracker)	O(P + O log U)	Copies P piece hashes and resolves O owners.
download (client)	O(S) hashing + network	Every byte is hashed twice: once per piece and once in the full-file pass.
snapshot sync	O(total metadata)	One linear pass.

Design parameters

- **Piece size 512 KB:** big enough that the cost of opening a TCP connection per piece is small, and small enough that a large file splits into many pieces that can be fetched in parallel. A failed piece only costs 512 KB to fetch again.
- **Up to 8 worker threads per download:** keeps several requests in flight to hide network latency, without starting an excessive number of threads. Small files use fewer workers (min(8, N)).
- **Memory:** each worker buffers at most one piece (512 KB), so one download uses at most about 4 MB of buffers. The full-file check streams in 512 KB blocks.
- **Pre-allocation:** `ftruncate` reserves the file size once, so later random-offset writes never have to grow the file.

15. Limitations and Assumptions

These points were found by reading the current code. They matter for understanding exactly how the system behaves.

1. **No automatic client reconnection.** After a tracker failover, clients connected to the old primary exit ("Connection closed by tracker"). They must be restarted to find the new primary. Saved seed lists make re-joining quick.
2. **Login state is not replicated.** `userSessions` is rebuilt only when users log in again on the new primary.
3. **Snapshot drops `FileMetadata.groupId`.** `deserializeSystemState` does not set `groupId` for files. If files that reached the backup through a *snapshot* (not through `UPDATE`s) are downloaded after a failover, the `OK DOWNLOAD` reply has an empty group field. The client then parses the reply at the wrong positions and the download fails. Setting `fm.groupId = gid` during deserialisation would fix this.
4. **Passwords are stored in plain text.** The field is called `passwordHash`, but the raw password is stored and also sent in `UPDATE CREATE_USER` sync messages.
5. **Tracker does not stop a second session for the same user.** The client blocks two logins in one process. But if the same user logs in from two processes, the latest login overwrites the seeder address.
6. **Seeder lookup is keyed by file name only.** `userSeeds[user][fileName]` ignores the group, so two different files with the same name in different groups collide on one peer.
7. **`dest_path` is a file path.** `download_file g f dest` opens `dest` as the output file itself. Give a full file path such as `/tmp/f.txt`, not a directory.
8. **Empty files cannot be shared.** The tracker rejects an upload with `fileSize = 0`.
9. **Re-seed reply race.** The re-seed thread reads the tracker's reply on the same socket that the shell uses. If the user issues a command at that moment, the two replies can reach the wrong reader.
10. **`stopShare` does not take `stateMutex`.** Every other `SystemState` operation does.
11. **The update log is never trimmed.** `updateLog` grows for the lifetime of the primary. `replayLog()` exists but is never called, because a new backup always gets a full snapshot instead.
12. **Names cannot contain spaces.** The protocol splits on whitespace, so user IDs, group IDs, file names and paths must not contain spaces.
13. **No encryption.** All traffic, including passwords, is plain text over TCP.
14. **Exactly two trackers are assumed**, and each client's seeder `IP:PORT` must be reachable by the other peers.

16. Command Reference

Command	Needs login	Who	Effect
`create_user <uid> <pwd>`	No	Anyone	Registers an account.
`login <uid> <pwd>`	No	Anyone	Starts the seeder, opens a session, and restores saved seeds.
`create_group <gid>`	Yes	Any user	Creates a group, with the caller as owner.
`list_groups`	Yes	Any user	Lists all groups.
`join_group <gid>`	Yes	Non-member	Adds a pending join request.
`list_requests <gid>`	Yes	Owner	Shows pending requests.
`accept_request <gid> <uid>`	Yes	Owner	Makes the requester a member.
`leave_group <gid>`	Yes	Member	Leaves the group, removes the user's shares there, and passes ownership on if needed.
`upload_file <gid> <path>`	Yes	User	Hashes the file, publishes its metadata, and starts seeding it.
`list_files <gid>`	Yes	User	Lists the files shared in the group.
`download_file <gid> <file> <dest_file_path>`	Yes	User	Starts a background parallel download, then re-seeds the file.
`show_downloads`	Yes (local)	User	`[D]` in progress with piece count, `[C]` completed.
`stop_share <gid> <file>`	Yes	File owner	Stops seeding the file. If no owners remain, the file is removed from the group.
`logout`	Yes	User	Ends the session and stops the seeder.
`exit`	No	Anyone	Closes the client. The tracker logs the user out automatically.
tracker console: `quit` / `exit`	–	Operator	Stops the tracker process.

The code checks commands before the permissions listed above: a missing group, missing file or wrong password each returns an `ERR …` line with the reason.

17. Glossary

Term	Definition
SHA-1	A hash function that produces a 160-bit fingerprint (40 hex characters) of any data. Any change to the data changes the fingerprint.
Primary–backup replication	One server handles all requests and forwards each state change to a passive copy that can take over.
Operation-log replication	Replicating a system by sending the *operations* performed instead of the resulting data.

Snapshot	A serialised copy of the whole state, used to bring a new backup up to date.
Work queue / thread pool	A fixed set of threads that repeatedly take tasks (piece indices) from a shared queue.
`pread` / `pwrite`	POSIX calls that read or write at an explicit file offset without moving the file pointer. This makes them safe for parallel access.
`ftruncate`	Sets a file's length. Used here to pre-size the download file.
`SO_RCVTIMEO`	A socket option that makes `recv()` give up after a timeout.
Swarm	The set of peers that currently hold and serve a given file.

Prepared by **Nilkanta Karak** (Roll No. **2025201031**) for Advanced Operating Systems, Assignment 3, IIIT Hyderabad, Monsoon 2025.